

泰芯芯片 FLASH 各种常见问题 FAQ



保密等级	A	泰芯芯片 FLASH 各种常见问题 FAQ	文件编号	TX-0000
发行日期	2026-03-30		文件版本	V1.0

责任与版权

责任限制

本文档内容仅供参考。珠海泰芯半导体有限公司（以下简称“泰芯”）未对本文档信息的准确性或完整性作出任何明示或暗示的陈述或保证，且不对使用本文件信息所产生的后果承担任何责任。

泰芯保留在不另行通知对本文件进行修改、改进或其他变更的权利。

泰芯没有为客户产品的应用、设计验证提供协助的义务。客户应自行对其应用的设计、验证和测试负责，并确保其应用符合所有法律、法规及安全相关的要求。

如因客户未遵守本协议导致任何损害、费用、损失及/或责任，泰芯不承担任何责任；同时，客户应全额赔偿因客户未遵守本协议而给泰芯造成的任何损害、费用、损失及/或责任。

版权声明

未经泰芯书面同意，任何一方不得为商业目的修改、改编、变更、翻译或基于本文件创建衍生作品。

未经泰芯书面同意，任何一方不得向第三方披露或分发本文件中提及的任何部分或全部源代码、SDK、二进制文件和目标代码。

任何一方不得修改、反向工程、反汇编、反编译或以其他方式试图发掘任何非源代码部分的 SDK，包括但不限于预编译的二进制文件和目标代码。

此外，任何可能侵犯泰芯或其他知识产权所有者权利的行为也均被严格禁止。

对于实施上述侵权行为的主体，泰芯有权依据中华人民共和国法律或其他可能适用的法律、国际条约采取必要的法律措施，包括但不限于对侵权人提起诉讼或仲裁，或申请法律强制措施。

珠海泰芯半导体有限公司

2024 年 09 月 24 日



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园
港 11 栋 3 楼

版权所有 侵权必究

Copyright © 2026 by TaiXin All Rights Reserved

保密等级	A	泰芯芯片 FLASH 各种常见问题 FAQ	文件编号	TX-0000												
发行日期	2026-03-30		文件版本	V1.0												
修订记录 <table><tr><th>日期</th><th>版本</th><th>描 述</th><th>修订人</th></tr><tr><td>2026-03-30</td><td>V1.0</td><td>初始版本, SDK-VERSION: 41569</td><td>TX</td></tr><tr><td></td><td></td><td></td><td></td></tr></table>					日期	版本	描 述	修订人	2026-03-30	V1.0	初始版本, SDK-VERSION: 41569	TX				
日期	版本	描 述	修订人													
2026-03-30	V1.0	初始版本, SDK-VERSION: 41569	TX													
泰芯半导体 TaiXin Semiconductor 珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited 珠海市高新区港湾一号科创园 港 11 栋 3 楼 版权所有 侵权必究 Copyright © 2026 by TaiXin All Rights Reserved																

保密等级	A	泰芯芯片 FLASH 各种常见问题 FAQ	文件编号	TX-0000
发行日期	2026-03-30		文件版本	V1.0
目录 泰芯芯片 FLASH 各种常见问题 FAQ 1 1. Flash 启动问题 2 1.1. 生产阶段方法 2 1.2. 调试阶段方法 2 2. 擦写动态挂起恢复(SDK 默认启用)3 3. DDR 功能（TXW81x 不需要考虑） 4 4. XIP 功能（TXW81x 不需要考虑）4 5. 安全区 5 6. 动态写保护（TXW81x 不需要考虑）6 6.1. 关闭动态写保护 6 6.2. 开启动态写保护6				
		珠海泰芯半导体有限公司 Zhuhai TaiXin Semiconductor Co., Limited	珠海市高新区港湾一号科创园 港 11 栋 3 楼	
版权所有 侵权必究 Copyright © 2026 by TaiXin All Rights Reserved				

1. Flash 启动问题

支持可外挂 Flash Boot 启动的芯片，常常会出现一样的固件，换外置 Flash 后无法正常 Boot 启动。此类问题通常是由于 Flash 四线使能引起的。下文中 1.1 节的方法是更推荐的，1.2 节介绍了常见的四线使能配置。

- 前置内容：

一般而言，泰芯芯片通过 Flash Boot 启动，固件在编译完成后会自动被添加一段固件头信息。芯片启动时会根据固件头信息初始化 Flash(以发送指令码的方式)，这些指令码可以在 makecode.ini 中进行配置。

1.1. 生产阶段方法

通过修改 makecode.ini 可以完成 flash4 线使能，但如果需要一份固件兼容不同型号 flash 的话，单纯修改 makecode.ini 的方式就不适用了，我们推荐使用通用烧录器，在 flash 贴片前先烧录对应固件，并且设置开启 QE BIT。

对应固件需要在 makecode.ini 中关闭 SpecSequenceEn。

```
WireMode4En=1                ; 4wire mode enable[0,1]
SpecSequenceEn=0              ;
SpecSequenceNumbers=0
; spec cmd sequence (format: cmd + dummy + data_lens + data) numbers
```

1.2. 调试阶段方法

- Makecode.ini 切片 1:

```
WireMode4En=1                ; 4wire mode enable[0,1]
SpecSequenceEn=1              ;
SpecSequenceNumbers=6
; spec cmd sequence (format: cmd + dummy + data_lens + data) numbers
SpecSequence0=50000000        ; spec cmd "write enable "
```

```
SpecSequence1=0100020002      ; spec cmd "qual mode enable for winbond "
SpecSequence2=05800101        ; spec cmd "wait busy "
SpecSequence3=50000000        ; spec cmd "write enable "
SpecSequence4=31000102        ; spec cmd "qual mode enable for winbond "
SpecSequence5=05800101        ; spec cmd "wait busy "
```

- **Makecode.ini 切片 2:**

```
WireMode4En=1                  ; 4wire mode enable[0,1]
SpecSequenceEn=1               ;
SpecSequenceNumbers=6
; spec cmd squence (format: cmd + dummy + data_lens + data) numbers
SpecSequence0=06000000        ; spec cmd "write enable "
SpecSequence1=0100020002      ; spec cmd "qual mode enable for winbond "
SpecSequence2=05800101        ; spec cmd "wait busy "
SpecSequence3=06000000        ; spec cmd "write enable "
SpecSequence4=31000102        ; spec cmd "qual mode enable for winbond "
SpecSequence5=05800101        ; spec cmd "wait busy "
```

- **Makecode.ini 切片 3:**

```
WireMode4En=1                  ; 4wire mode enable[0,1]
SpecSequenceEn=1               ;
SpecSequenceNumbers=3
; spec cmd squence (format: cmd + dummy + data_lens + data) numbers
SpecSequence0=06000000        ; spec cmd "write enable "
SpecSequence1=01000140        ; spec cmd "qual mode enable for winbond "
SpecSequence3=05800101        ; spec cmd "wait busy "
```

以上是三种典型的 flash 4 线使能配置，目前暂未看到更多例外。通常逐一尝试以上三种配置即可让 flash 4 线启动。

2. 擦写动态挂起恢复(SDK 默认启用)

由于程序运行在 Flash 上，而擦写时读 Flash 会出现读到 0 情况，因此在擦写 Flash 过程中无法运行代码，为了防止跑飞还需要关闭中断。因此在擦写时会出现几十到几百毫秒

的关中断时间，一部分程序可能因此出现问题。为了解决这个问题，SDK 引入了擦写动态挂起的机制，可以配置 tsr 时间内自动挂起擦写状态和擦写线程，挂起后系统即可处理其他紧急的事情，trs 时间后擦写线程才会恢复到 ready 状态。

- 1) 大部分Flash支持擦写suspend和resume功能, 客户可以配置tsr,trs,tsus,trus满足对应flash所需要的时序要求。
- 2) 对于不支持suspend和resume的flash, 客户可以配置device.c/spi7.wip.tsr=60000解决。
 - Tsr:擦除开始到擦除挂起的最大时间(ms)
 - Trs:擦除挂起到擦除恢复必要的间隔时间(ms)
 - Tsus:擦除挂起指令到下一条指令, cs必要的拉高时间(us)
 - Trus:擦除恢复指令到下一条指令, cs必要的拉高时间(us)

这些参数在 FLASH SPEC 中 AC Characteristics 中会描述, 可能有一部分是没有的, 大部分时候都不需要修改这些参数

SDK 默认值:

Tsr=8,trs=4,tsus=20,trus=20

3. DDR 功能 (TXW81x 不需要考虑)

TXW82x 芯片支持 DDR Flash, SDK 默认不启用 DDR 功能, 支持 DDR 的 Flash 比较少, 如果 Flash 支持 DDR, 可以在 device.c 中, spi7.ddd=1,启动 DDR 功能。DDR 目前最高只支持到 80Mhz 的频率。

4. XIP 功能 (TXW81x 不需要考虑)

TXW82x 芯片支持 Flash XIP, SDK 默认开启 XIP 功能, 大部分 Flash 支持 XIP 功能, XIP 能够让每一次读取访问都可以少一个指令码, 在 4 线模式下, 开启 XIP, 在 SDR

模式能减小 16.6%访问时间，在 DDR 模式下能减小 25%的访问时间。

在 41569 版本后，对 XIP 才有了比较完善的支持。在 device.c 中，spi7.xip=1/0,开关 XIP 功能。

5. 安全区

一些 Flash 可能会提供额外的一片存储空间用于存储 OTP 数据，通常称这块区域为安全区。安全区的读写会有点区别，在 Flash spec 上会描述它们安全区的访问地址，通常在第一页 FEATURE 会描述是否支持安全区，全局搜索 44H(安全区擦除指令)，指令码描述那里会说明安全区的地址。

7.26 Erase Security Registers (44H)

The GD25Q32E provides 3x1024-Byte Security Registers which can be erased and programmed individually. These registers may be used by the system manufacturers to store security and other important information separately from the main memory array.

The Erase Security Registers command is similar to Sector/Block Erase command. A Write Enable (WREN) command must previously have been executed to set the Write Enable Latch (WEL) bit.

The Erase Security Registers command sequence: CS# goes low → sending Erase Security Registers command → 3-byte address on SI → CS# goes high. The command sequence is shown below. CS# must be driven high after the eighth bit of the last address byte has been latched in; otherwise the Erase Security Registers command is not executed. As soon as CS# is driven high, the self-timed Erase Security Registers cycle (whose duration is t_{SE}) is initiated. While the Erase Security Registers cycle is in progress, the Status Register may be read to check the value of the Write in Progress (WIP) bit. The Write in Progress (WIP) bit is 1 during the self-timed Erase Security Registers cycle, and is 0 when it is completed. At some unspecified time before the cycle is completed, the Write Enable Latch (WEL) bit is reset. The Security Registers Lock Bit (LB1, LB2, LB3) in the Status Register can be used to OTP protect the security registers. Once the LB bit is set to 1, the Security Registers will be permanently locked; the Erase Security Registers command will be ignored.

Address	A23-16	A15-12	A11-10	A9-0
Security Register #1	00H	0001b	00b	Don't care
Security Register #2	00H	0010b	00b	Don't care
Security Register #3	00H	0011b	00b	Don't care

图 5-1 某款 flash 对于 44h 指令的描述

可以看到总共有 3 个安全区区域，地址分别是 0x001000,0x002000,0x003000,每一个区域由 10bit 寻址，大小 1kB。

客户需要访问到正确的地址才能正常读写安全区。安全区相关读写样例请参阅 `app/flash/flash_read_demo.c/flash_security_demo()`;

6. 动态写保护 (TXW81x 不需要考虑)

这个功能的实现思路是在非必要时, 让 Flash 处于写保护状态, 必要时(需要擦写 Flash), 短暂的关闭 Flash 写保护功能。由于兼容性问题, 默认不再打开这个功能。客户如有需要, 可以根据以下内容配置。

6.1. 关闭动态写保护

对于旧 SDK(版本号小于 41569), 如果你遇到了有关 Flash 写入失败的问题, 那大概率就是写保护不兼容导致的。你可以选择关闭动态写保护, 以下是关闭方法:

```
device.c/spi7->spi7.nbp=1; //nbp 赋值为 1  
hal/spi_nor.c/spi_nor_attach()->flash->bp_conv=NULL; //块保护逻辑
```

6.2. 开启动态写保护

为了提高兼容性, 需要根据不同的 Flash id 来选择不同的写保护策略。通常, 这需要你对 SPI Flash 的寄存器有一定的理解, 具体参阅对应 Flash 的数据手册中状态寄存器/写保护章节。

下图是某 Flash 的 sr/wp area 的关系图, 每个 Flash spec 都有类似的图

Table 5. GD25Q32E Protected area size (CMP=1)

Status Register Content					Memory Content			
BP4	BP3	BP2	BP1	BP0	Blocks	Addresses	Density	Portion
X	X	0	0	0	ALL	000000H-3FFFFFFH	4MB	ALL
0	0	0	0	1	0 to 62	000000H-3EFFFFH	4032KB	Lower 63/64
0	0	0	1	0	0 to 61	000000H-3DFFFFH	3968KB	Lower 31/32
0	0	0	1	1	0 to 59	000000H-3BFFFFH	3840KB	Lower 15/16
0	0	1	0	0	0 to 55	000000H-37FFFFH	3584KB	Lower 7/8
0	0	1	0	1	0 to 47	000000H-2FFFFFFH	3MB	Lower 3/4
0	0	1	1	0	0 to 31	000000H-1FFFFFFH	2MB	Lower 1/2
0	1	0	0	1	1 to 63	010000H-3FFFFFFH	4032KB	Upper 63/64
0	1	0	1	0	2 to 63	020000H-3FFFFFFH	3968KB	Upper 31/32
0	1	0	1	1	4 to 63	040000H-3FFFFFFH	3840KB	Upper 15/16
0	1	1	0	0	8 to 63	080000H-3FFFFFFH	3584KB	Upper 7/8
0	1	1	0	1	16 to 63	100000H-3FFFFFFH	3MB	Upper 3/4
0	1	1	1	0	32 to 63	200000H-3FFFFFFH	2MB	Upper 1/2
X	X	1	1	1	NONE	NONE	NONE	NONE
1	0	0	0	1	0 to 63	000000H-3FEFFFFH	4092KB	L-1023/1024
1	0	0	1	0	0 to 63	000000H-3FDFFFFH	4088KB	L-511/512
1	0	0	1	1	0 to 63	000000H-3FBFFFFH	4080KB	L-255/256
1	0	1	0	X	0 to 63	000000H-3F7FFFFH	4064KB	L-127/128
1	0	1	1	0	0 to 63	000000H-3F7FFFFH	4064KB	L-127/128
1	1	0	0	1	0 to 63	001000H-3FFFFFFH	4092KB	U-1023/1024
1	1	0	1	0	0 to 63	002000H-3FFFFFFH	4088KB	U-511/512
1	1	0	1	1	0 to 63	004000H-3FFFFFFH	4080KB	U-255/256
1	1	1	0	X	0 to 63	008000H-3FFFFFFH	4064KB	U-127/128
1	1	1	1	0	0 to 63	008000H-3FFFFFFH	4064KB	U-127/128

图 6-2-1 一种 SR（状态寄存器）和 memory area 的关系

- 相关控制代码:

Spi_nor.c/spi_nor_attach():

“flash->bp_conv(struct spi_nor_flash*flash,uint32_t addr1,uint32_t addru,struct bpreg_cfg*bp)”负责将[addr1.addrh]转化为寄存器配置描述 struct bpreg_cfg,写保护逻辑自动在关键时刻把 struct bpreg_cfg 寄存器配置写入到 Flash 中。

目前实现了“xip_nor_flash_unbp_convert_tbl()”。即

flash->bp_conv=xip_nor_flash_unbp_convert_tbl;xip_nor_flash_unbp_convert_tbl 通过 struct bp_area 将[addr1.addrh]转化为 struct bpreg_cfg。

struct bp_area 结构体用于描述寄存器配置及其对应写保护区域的关系。

struct bp_area 由宏 DEF_BP_AREA_TB_BPV(tb,bpv,addr1,addrh)描述, tb 和 bpv 都是 Flash 寄存器中的写保护控制域, tb 通常只有 1 个 bit, bpv 可能有 4 个或 5 个 bit。在图 6-2-1 中 bp3 也可以称为 tb(top/button), 它控制着 portion 项是 Upper 或是

Lower。

每一条 `DEF_BP_AREA_TB_BPV()` 都描述了 `[addrl,addrh]` 区域对应的写保护寄存器配置。客户只需要根据不同的 flash，配置不同的 `struct bp_area` 即可，其余由软件自动控制。lib/bus/spi/spi_nor/spi_nor_bus.c 中 `const struct bp_area bp_area_tbl_2MB{...};` 是个典型例子。